

Huang-Chen Notes

James Evans

08/20/2025

Universal Flows For a general learning framework to be meaningful, a model needs to be flexible enough to capture variations in the data distribution. In the context of density modeling, this corresponds to the model’s capability to represent arbitrary probability distributions of interest. Even though there exists a long history of literature on universal approximation capability of deep neural networks (Cybenko, 1989; Lu et al., 2017; Lin & Jegelka, 2018), invertible neural networks generally have limited expressivity and cannot approximate arbitrary functions. However, for the purpose of approximating a probability distribution, it suffices to show that the distribution induced by a normalizing flow is universal.

Among many ways to establish distributional universality of flow based methods (e.g. Huang et al. 2018; 2020b; Teshima et al. 2020; Kong & Chaudhuri 2020), one particular approach is to approximate a *deterministic coupling* between probability measures. Given a pair of probability densities p_X and p_Y , a deterministic coupling is a mapping g such that $g(X) \sim p_Y$ if $X \sim p_X$. We seek to find a coupling that is invertible, or at least can be approximated by invertible mappings.

Universal Flows

For a density model to be useful, it must be flexible enough to represent any probability distribution. In normalizing flows, this corresponds to the ability to transform a simple base distribution into an arbitrary target distribution using an invertible neural network.

While standard neural networks are universal approximators of functions, invertible networks cannot directly approximate all functions. However, it is sufficient to show that they can approximate any probability distribution. This is done using the idea of a *deterministic coupling*.

Given two probability distributions p_X and p_Y , a deterministic coupling is a mapping g such that

$$X \sim p_X \implies g(X) \sim p_Y.$$

In other words, if we draw a sample from p_X and apply g , the output follows p_Y . Normalizing flows learn such an invertible mapping g , or an arbitrarily good approximation of it. This establishes the **universality** of flows: they can represent any probability distribution.

Why Invertibility is Required

Normalizing flows rely on the change-of-variable formula for densities:

$$p_Y(y) = p_X(f^{-1}(y)) \left| \det \frac{\partial f^{-1}(y)}{\partial y} \right|.$$

This formula only holds if f is invertible, since otherwise the pre-image $f^{-1}(y)$ would be undefined or ambiguous. Invertibility is essential for three reasons:

- It guarantees a one-to-one correspondence between samples: each y comes from exactly one x .
- It makes density computation possible, since we must trace a point y back to x through f^{-1} .
- It allows both sampling ($y = f(x)$) and likelihood evaluation ($x = f^{-1}(y)$) to be performed consistently.

Flows do two jobs

A normalizing flow isn't just for **sampling** or just for **densities** — it needs to do both:

1. Sampling (forward direction):

- We start with an easy base distribution $X \sim p_X$ (e.g. Gaussian).
- Apply the map f to get samples:

$$y = f(x), \quad x \sim p_X.$$

- This gives us realistic synthetic data from p_Y .

2. Likelihood evaluation (backward direction):

- Given a data point y , we want to compute its likelihood under the model.
- To do this, we must map it back:

$$x = f^{-1}(y),$$

and then use the change-of-variable formula

$$p_Y(y) = p_X(x) \left| \det \frac{\partial f^{-1}(y)}{\partial y} \right|.$$

A natural way to ensure invertibility is to require convexity. By Brenier's Theorem, if f is the gradient of a convex potential function, then f is invertible almost everywhere. Thus, convexity provides the mathematical guarantee of invertibility that normalizing flows depend on.

Brenier's Theorem: Optimality vs. Invertibility

Brenier's Theorem (for quadratic-cost optimal transport) states that if ρ and μ are probability measures on $\mathbb{R}^d$, with ρ absolutely continuous, then the optimal transport map T pushing ρ to μ exists and has the form

$$T(x) = \nabla \varphi(x),$$

where φ is a convex function.

The theorem is primarily an **optimality result**: it characterizes the *optimal* transport map as the gradient of a convex potential. However, convexity also implies an important structural property:

- If $T = \nabla \varphi$ with φ convex, then T is monotone (in fact, cyclically monotone).
- Monotonicity guarantees that T is injective almost everywhere, which ensures the existence of a well-defined inverse T^{-1} except on a set of measure zero.

Thus, convexity provides not only the **optimality** of the transport map, but also the **invertibility** (almost **everywhere**) that normalizing flows rely on.

Some recent works are also inspired by Brenier’s theorem and utilize a convex potential to parameterize a critic model, starting from Taghvaei & Jalali (2019), and further built upon by Makkuva et al. (2019) who parameterize a generator with a convex potential and concurrently by Korotin et al. (2019). Our work sets itself apart from these prior works in that it is entirely likelihood-based, minimizing the (empirical) KL divergence as opposed to an approximate optimal transport cost.

Relation to Prior Work

Some prior works, inspired by Brenier’s theorem, parameterize the transport map as the gradient of a convex potential and then optimize an approximation to the quadratic optimal transport cost. Notable examples include Taghvaei & Jalali (2019), Makkuva et al. (2019), and Korotin et al. (2019).

In contrast, the present work does not minimize the OT cost. Instead, it adopts a purely likelihood-based approach: the flow is trained by minimizing the (empirical) KL divergence between the data distribution and the model. This distinguishes it from OT-based methods, while still leveraging convex potentials for parameterization.

3 CONVEX POTENTIAL FLOWS

Given a strictly convex potential F , we can define an injective map (invertible from its image) via its gradient $f = \nabla F$, since the Jacobian of f is the Hessian matrix of F , and is thus positive definite. In this section, we discuss the parameterization of the convex potential F (3.1), and then address gradient estimation for CP-Flows (3.2). We examine the connection to other parameterization of normalizing flows (3.3), and finally rigorously prove universality in the next section.

Defining the Transport Map

From the convex potential F , we define the transport map

$$f(x) = \nabla F(x).$$

This means the transport (or flow) is given by the **gradient of the convex potential**.

Why is it Invertible?

- The **Jacobian of f** is the **Hessian of F** :

$$J_f(x) = \nabla^2 F(x).$$

- Since F is **strictly convex**, its Hessian is **positive definite**.
- A positive definite Jacobian ensures that f is **injective** (one-to-one, hence invertible onto its image).

3.1 MODELING

Input Convex Neural Networks We use $L(x)$ to denote a linear layer, and $L^+(x)$ to denote a linear layer with positive weights. We use the (fully) input-convex neural network (ICNN, [Amos et al. \(2017\)](#)) to parameterize the convex potential, which has the following form

$$F(x) = L_{K+1}^+(s(z_K)) + L_{K+1}(x) \quad z_k := L_k^+(s(z_{k-1})) + L_k(x) \quad z_1 := L_1(x)$$

where s is a non-decreasing, convex activation function. In this work, we use softplus-type activation functions, which is a rich family of activation functions that can be shown to uniformly approximate the ReLU activation. See Appendix B for details.

Input Convex Neural Networks

To parameterize the convex potential, we employ the *input convex neural network*. The ICNN is a special neural network architecture that enforces convexity of the output with respect to its inputs by design.

Notation

- $L(x)$ denotes a standard linear layer of the form $L(x) = Wx + b$.
- $L^+(x)$ denotes a linear layer where the weight matrix W has *nonnegative* entries, i.e. $W \geq 0$ elementwise.
- $s(\cdot)$ is a convex, nondecreasing activation function.

Network Structure

The convex potential $F(x)$ is constructed recursively as follows:

$$F(x) = L_{K+1}^+(s(z_K)) + L_{K+1}(x),$$

where the hidden states $\{z_k\}$ are defined layer by layer:

$$z_1 := L_1(x),$$

$$z_k := L_k^+(s(z_{k-1})) + L_k(x), \quad k = 2, \dots, K.$$

Example

To see explicitly why convexity is preserved, consider the second hidden layer z_2 . By definition,

$$z_2 = L_2^+(s(z_1)) + L_2(x).$$

First, the base case is

$$z_1 = L_1(x),$$

which is a linear function of x . Since all linear functions are convex, z_1 is convex in x . Next, $s(z_1)$ is obtained by applying an activation $s(\cdot)$ (for example, $\max\{0, z_1\}$ in the case of ReLU). Because z_1 is convex and $s(\cdot)$ is convex and nondecreasing, the composition $s(z_1)$ is convex. Applying L_2^+ (a linear map with nonnegative weights) to $s(z_1)$ preserves convexity. This follows from the fact that convex functions are closed under nonnegative linear combinations. Finally, $L_2(x)$ is a purely linear term in x , hence convex.

Therefore,

$$z_2 = L_2^+(s(z_1)) + L_2(x)$$

is the sum of two convex functions, and is therefore convex.

This same reasoning extends inductively. Suppose z_{k-1} is convex. Then $s(z_{k-1})$ is convex (convex + nondecreasing composition), $L_k^+(s(z_{k-1}))$ is convex (nonnegative linear combination), and $L_k(x)$ is convex (linear). Their sum, z_k , is convex.

By induction, all hidden layers z_k are convex in x , and thus the final potential $F(x)$ is convex in x .

Invertibility and Inversion Procedure If the activation s is twice differentiable, then the Hessian H_F is positive semi-definite. We can make it strongly convex by adding a quadratic term $F_\alpha(x) = \frac{\alpha}{2}\ x\ _2^2 + F(x)$, such that $H_{F_\alpha} \succeq \alpha I \succ 0$. This means the gradient map $f_\alpha = \nabla F_\alpha$ is injective onto its image. Furthermore, it is surjective since for any $y \in \mathbb{R}^d$, the potential $x \mapsto F_\alpha(x) - y^\top x$ has a unique minimizer¹ satisfying the first order condition $\nabla F_\alpha(x) = y$, due to the strong convexity and differentiability. We refer to this invertible mapping f_α as the <i>convex potential flow</i>, or the CP-Flow. The above discussion also implies we can plug in a black-box convex solver to invert the gradient map f_α, which we summarize in Algorithm 1. Inverting a batch of independent inputs is as simple as summing the convex potential over all inputs: since all of the entries of the scalar l in the mini-batch are independent of each other, computing the gradient all l's wrt all x's amounts to computing the gradient of the summation of l's wrt all x's. Due to the convex nature of the problem, a wide selection of algorithms can be used with convergence guarantees (Nesterov, 1998). In practice, we use the <i>L-BFGS</i> algorithm (Byrd et al., 1995) as our CvxSolver.	Algorithm 1 Inverting CP-Flow. <pre> 1: procedure INVERT($F, y, \text{CvxSolver}$) 2: Initialize $x \leftarrow y$ 3: def closure(): 4: Compute loss: $l \leftarrow F(x) - y^\top x$ 5: return l 6: $x \leftarrow \text{CvxSolver}(\text{closure}, x)$ 7: return x </pre>
---	--

Invertibility and Convex Potential Flow

Consider the convex potential $F(x)$ parameterized by an ICNN. Its Hessian $H_F = \nabla^2 F(x)$ is positive semidefinite, which guarantees convexity but not strong convexity. To enforce strong convexity, we add a quadratic term:

$$F_\alpha(x) = \frac{\alpha}{2}\|x\|^2 + F(x), \quad \alpha > 0.$$

The resulting Hessian is

$$H_{F_\alpha} = H_F + \alpha I \succeq \alpha I \succ 0,$$

which ensures that F_α is strictly convex everywhere. Strong convexity implies that the gradient map

$$f_\alpha(x) = \nabla F_\alpha(x)$$

is a *bijection*:

- **Injective (one-to-one):** two distinct inputs $x_1 \neq x_2$ cannot map to the same output.
- **Surjective (onto):** for any $y \in \mathbb{R}^d$, there exists an x such that $\nabla F_\alpha(x) = y$.

When solving a convex optimization problem

$$\min_x \phi(x),$$

with $\phi(x)$ differentiable and convex, the necessary and sufficient condition for x^* to be the minimizer is that the gradient vanishes:

$$\nabla \phi(x^*) = 0.$$

This is called the *first-order optimality condition*.

In the CP-Flow setting, we consider

$$\phi(x) = F_\alpha(x) - y^\top x,$$

where F_α is strongly convex. Taking the gradient gives

$$\nabla\phi(x) = \nabla F_\alpha(x) - y.$$

Applying the first-order optimality condition, we set this equal to zero:

$$\nabla F_\alpha(x^*) = y.$$

Thus, inverting the gradient map amounts to solving this convex optimization problem. Since F_α is strongly convex, a unique minimizer x^* exists, and the inversion is always well-defined.

We call this invertible mapping $f_\alpha = \nabla F_\alpha$ the *convex potential flow (CP-Flow)*. Instead of relying on a closed-form inverse, the inversion can be computed by solving the convex problem

$$x^* = \arg \min_x (F(x) - y^\top x),$$

which can be handled by any black-box convex solver. Because of convexity, standard optimization methods such as L-BFGS or accelerated gradient descent enjoy guaranteed convergence.

For a batch of independent targets $\{y_i\}_{i=1}^n$, the inversion problem decouples across samples. We can minimize

$$\sum_{i=1}^n (F(x_i) - y_i^\top x_i)$$

with respect to all $\{x_i\}$, and the gradients can be computed in parallel. This makes CP-Flow efficient to invert even in large-scale settings.

Estimating Log Probability Following equation (1), computing the log density for CP-Flows requires taking the log determinant of a symmetric positive definite Jacobian matrix (as it is the Hessian of the potential). There exists numerous works on estimating spectral densities (e.g. [Tal-Ezer & Kosloff, 1984](#); [Silver & Röder, 1994](#); [Han et al., 2018](#); [Adams et al., 2018](#)), of which this quantity is a special case. See [Lin et al. \(2016\)](#) for an overview of methods that only require access to Hessian-vector products. Hessian-vector products (hvp) are cheap to compute with reverse-mode automatic differentiation ([Baydin et al., 2017](#)), which does not require constructing the full Hessian matrix and has the same asymptotic cost as evaluating F_α .

In particular, the log determinant can be rewritten in the form of a generalized trace $\text{tr} \log H$. [Chen et al. \(2019a\)](#) limit the spectral norm (i.e. eigenvalues) of H and directly use the Taylor expansion of the matrix logarithm. Since our H has unbounded eigenvalues, we use a more complex algorithm designed for symmetric matrices, the *stochastic Lanczos quadrature* (SLQ; [Ubaru et al., 2017](#)). At the core of SLQ is the Lanczos method, which computes m eigenvalues of H by first constructing a symmetric tridiagonal matrix $T \in \mathbb{R}^{m \times m}$ and computing the eigenvalues of T . The Lanczos procedure only requires Hessian-vector products, and it can be combined with a stochastic trace estimator to provide a stochastic estimate of our log probability. We chose SLQ because it has shown theoretically and empirically to have low variance ([Ubaru et al., 2017](#)).

Stochastic Lanczos Quadrature (SLQ)

Our goal is to compute

$$\log \det H = \text{tr}(\log H),$$

where H is a large symmetric positive definite matrix. Direct computation is infeasible, so we combine two techniques: stochastic trace estimation and the Lanczos algorithm.

- **SPD case (yours):** If H is symmetric positive definite, it has eigen-decomposition $H = V \text{diag}(\lambda_1, \dots, \lambda_n) V^\top$ with $\lambda_i > 0$.

Then

$$\log H = V \text{diag}(\log \lambda_1, \dots, \log \lambda_n) V^\top,$$

so

$$\text{tr}(\log H) = \sum_i \log \lambda_i = \log \left(\prod_i \lambda_i \right) = \log \det H.$$

Hence the identity holds exactly and is real-valued.

Stochastic Trace Estimation

For a random probe vector z (Rademacher), we have

$$\mathbb{E}[z^\top f(H) z] = \text{tr}(f(H)).$$

Thus, for $f(H) = \log H$, the trace can be approximated as

$$\text{tr}(\log H) \approx \frac{1}{k} \sum_{i=1}^k z_i^\top \log(H) z_i,$$

where $\{z_i\}_{i=1}^k$ are independent random vectors.

Lanczos Reduction

The key challenge in evaluating $z^\top \log(H) z$ is that H is an $n \times n$ matrix with n potentially in the thousands or millions. Direct methods that form H or compute its eigen-decomposition are infeasible. The Lanczos algorithm overcomes this by reducing the problem to a much smaller matrix.

Krylov Subspace Construction

Starting from the probe vector z , we repeatedly apply H to build the Krylov subspace

$$\mathcal{K}_m(H, z) = \text{span}\{z, Hz, H^2z, \dots, H^{m-1}z\}.$$

This subspace captures how H acts on z and typically has dimension $m \ll n$.

Probe vectors

Their entries are independent, with mean 0 and variance 1.

- **Rademacher vector:** each entry is randomly $+1$ or -1 with equal probability.
- **Gaussian vector:** each entry sampled from $\mathcal{N}(0, 1)$.

Note: You usually use several probe vectors $(z_1, z_2, \dots, z_k)$ to reduce variance.

We use Gram–Schmidt orthogonalization (or a stable variant) to build an orthonormal basis

$$Q_m = [q_1, q_2, \dots, q_m] \in \mathbb{R}^{n \times m}.$$

Gram–Schmidt Orthogonalization

Given a set of linearly independent vectors

$$v_1, v_2, \dots, v_m \in \mathbb{R}^n,$$

the Gram–Schmidt procedure constructs an orthonormal basis

$$q_1, q_2, \dots, q_m,$$

such that $q_i^\top q_j = 0$ for $i \neq j$ and $\|q_j\| = 1$.

Step-by-Step Process

(1) **First vector:** Normalize the first vector:

$$q_1 = \frac{v_1}{\|v_1\|}.$$

(2) **Inductive step:** For the next vector v_j , subtract off its projection onto all previous q_i :

$$u_j = v_j - \sum_{i=1}^{j-1} (q_i^\top v_j) q_i.$$

Explanation: The term $(q_i^\top v_j) q_i$ is the projection of v_j onto q_i because:

- $q_i^\top v_j$ is the scalar component of v_j in the direction of q_i (dot product).
- Multiplying by q_i gives the vector lying along q_i with exactly that magnitude.

By subtracting these projections for all $i < j$, the residual u_j is guaranteed to be orthogonal to $q_1, \dots, q_{j-1}$.

(3) **Normalize:**

$$q_j = \frac{u_j}{\|u_j\|}.$$

(4) **Repeat:** Continue until all $q_1, \dots, q_m$ are constructed.

Where the Vectors Come from in Lanczos

- We start with the **probe vector** z .
- Then we generate a sequence by repeatedly applying the matrix H :

$$v_1 = z, \quad v_2 = Hz, \quad v_3 = H^2z, \quad \dots, \quad v_m = H^{m-1}z.$$

Linear independence issue

- In general, the vectors $\{z, Hz, H^2z, \dots\}$ are **not guaranteed** to be linearly independent.
- But in practice: For a “generic” random probe vector z , the sequence tends to be independent for many steps, especially if H has a spread of eigenvalues. When dependence occurs (*breakdown*), the Lanczos process stops early.

Gram–Schmidt’s role

- Gram–Schmidt is applied to this sequence $\{z, Hz, H^2z, \dots\}$.
- It produces an **orthonormal basis**:

$$Q_m = [q_1, q_2, \dots, q_m],$$

where each q_j is orthogonal to the others and has unit length.

Tridiagonal Projection

By construction, the matrix representation of H in the Q_m basis is symmetric and tridiagonal:

$$T_m = Q_m^\top H Q_m = \begin{bmatrix} \alpha_1 & \beta_1 & 0 & \cdots & 0 \\ \beta_1 & \alpha_2 & \beta_2 & \ddots & \vdots \\ 0 & \beta_2 & \alpha_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{m-1} \\ 0 & \cdots & 0 & \beta_{m-1} & \alpha_m \end{bmatrix} \in \mathbb{R}^{m \times m}.$$

T_m is much smaller than H and retains essential spectral information: its eigenvalues approximate the extreme eigenvalues of H , and more generally, T_m provides an accurate compressed representation of how H acts on z . Since T_m is only $m \times m$ and tridiagonal, applying matrix functions like $\log(\cdot)$ is computationally cheap.

Change of Coordinates with Q_m

Let $Q_m = [q_1, \dots, q_m] \in \mathbb{R}^{n \times m}$ have orthonormal columns ($Q_m^\top Q_m = I_m$). Vectors in the Krylov subspace are represented by *coordinates* $y \in \mathbb{R}^m$ relative to the basis $\{q_1, \dots, q_m\}$.

Steps:

(1) **Coordinates $\rightarrow$ original space:**

$$x = Q_m y.$$

(2) **Apply the big operator:**

$$Hx.$$

(3) **Original space $\rightarrow$ coordinates (project onto the Q_m -basis):**

$$y' = Q_m^\top (Hx).$$

Why multiplying by Q_m is a change of basis

Let $Q_m = [q_1, \dots, q_m] \in \mathbb{R}^{n \times m}$ have orthonormal columns ($Q_m^\top Q_m = I_m$). The columns $\{q_i\}$ form a basis of the Krylov subspace.

Coordinates $\rightarrow$ Original Space

Given subspace coordinates $y \in \mathbb{R}^m$,

$$x = Q_m y = \sum_{i=1}^m y_i q_i.$$

This just forms the linear combination of basis vectors with coefficients y_i .

Original Space $\rightarrow$ Coordinates (Inside the Subspace)

If x lies in the span of $\{q_i\}$, its coordinates are obtained by

$$y = Q_m^\top x.$$

Why this works: writing $x = \sum_{j=1}^m y_j q_j$ and using orthonormality $q_i^\top q_j = \delta_{ij}$,

$$(Q_m^\top x)_i = q_i^\top x = q_i^\top \left(\sum_{j=1}^m y_j q_j \right) = \sum_{j=1}^m y_j (q_i^\top q_j) = y_i,$$

so $Q_m^\top x$ exactly recovers the coefficient vector y . If x is not exactly in the subspace, $y = Q_m^\top x$ gives the coordinates of the *orthogonal projection* of x onto the subspace:

$$\hat{x} = Q_m y = Q_m Q_m^\top x,$$

and $\hat{x}$ is the unique vector in $\text{span}\{q_i\}$ minimizing $\|x - \hat{x}\|_2$.

Step-by-step:

1. Distribute $q_i^\top$ over the sum (matrix mult. distributes over addition):

$$q_i^\top \left(\sum_{j=1}^m y_j q_j \right) = \sum_{j=1}^m q_i^\top (y_j q_j).$$

2. Pull out the scalar y_j (scalars commute with matrix/vector products):

$$\sum_{j=1}^m q_i^\top (y_j q_j) = \sum_{j=1}^m y_j (q_i^\top q_j).$$

- Q_m is the matrix whose **columns** are the basis vectors:

$$Q_m = [q_1 \ q_2 \ \cdots \ q_m].$$

- Therefore $Q_m^\top$ has those vectors as **rows**:

$$Q_m^\top = \begin{bmatrix} q_1^\top \\ q_2^\top \\ \vdots \\ q_m^\top \end{bmatrix}.$$

So when you multiply by x ,

$$Q_m^\top x = \begin{bmatrix} q_1^\top x \\ q_2^\top x \\ \vdots \\ q_m^\top x \end{bmatrix}.$$

The i -th entry of this vector is exactly $q_i^\top x$.

Formally:

$$(Q_m^\top x)_i = e_i^\top (Q_m^\top x) = (Q_m e_i)^\top x = q_i^\top x,$$

because $Q_m e_i$ selects the i -th **column** of Q_m , which is q_i .

Quadratic Form Approximation

The key identity is

$$z^\top f(H) z \approx \|z\|^2 e_1^\top f(T_m) e_1,$$

where $e_1 = (1, 0, \dots, 0)^\top$. Since T_m is tridiagonal and of size $m \times m$ with $m \ll n$, computing $\log(T_m)$ is cheap.

Final Estimator

Repeating the procedure with multiple random vectors yields the SLQ estimator:

$$\log \det H \approx \frac{1}{k} \sum_{i=1}^k \|z_i\|^2 e_1^\top \log(T_m^{(i)}) e_1.$$

This method only requires Hessian-vector products, avoids constructing H explicitly, and provides a low-variance stochastic estimate of the log determinant.

Applying the Logarithm to a Matrix

When we write $\log(T_m)$, we mean the **matrix logarithm**, defined through the eigen-decomposition of the symmetric tridiagonal matrix T_m . Since T_m is symmetric, it admits an eigen-decomposition

$$T_m = V \Lambda V^\top,$$

where V is an orthogonal matrix ($V^\top V = I$) and

$$\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_m)$$

contains the eigenvalues of T_m . For any scalar function f , the corresponding matrix function is defined as

$$f(T_m) = V f(\Lambda) V^\top,$$

where

$$f(\Lambda) = \text{diag}(f(\lambda_1), f(\lambda_2), \dots, f(\lambda_m)).$$

In particular, the matrix logarithm is

$$\log(T_m) = V \text{diag}(\log \lambda_1, \log \lambda_2, \dots, \log \lambda_m) V^\top.$$

Why this is Useful

The eigenvalues of T_m approximate those of H , and applying the logarithm means taking the log of each eigenvalue. Since $T_m \in \mathbb{R}^{m \times m}$ with $m \ll n$, computing its eigenvalues is cheap. This makes it feasible to evaluate expressions such as

$$z^\top \log(H) z \approx \|z\|^2 e_1^\top \log(T_m) e_1.$$

3.2 $\mathcal{O}(1)$ -MEMORY UNBIASED $\nabla \log \det H$ ESTIMATOR

We would also like to have an estimator for the *gradient* of the log determinant to enable variants of stochastic gradient descent for optimization. Unfortunately, directly backpropagating through the log determinant estimator is not ideal. Two major drawbacks of directly differentiating through SLQ are that it requires (i) differentiating through an eigendecomposition routine and (ii) storing all Hessian-vector products in memory (see fig. 2). Problem (i) is more specific to SLQ, because the gradient of an eigendecomposition is not defined when the eigenvalues are not unique (Seeger et al., 2017). Consequently, we have empirically observed that differentiating through SLQ can be unstable, frequently resulting in NaNs due to the eigendecomposition. Problem (ii) will hold true for other algorithms that also estimate $\log \det H$ with Hessian-vector products, and generally the only difference is that a different numerical routine would need to be differentiated through. Due to these problems, we do not differentiate through SLQ, but we still use it as an efficient method for monitoring training progress.

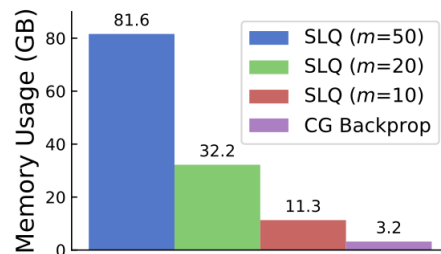


Figure 2: Memory for training CIFAR-10.

Connection to Log-Determinant Estimators in the Literature

Section 3.2 of the paper discusses the problem of estimating

$$\nabla \log \det H,$$

the gradient of the Hessian log-determinant. This quantity is introduced so it can be used directly inside stochastic gradient descent (SGD) updates.

Why We Do Not Require $\nabla \log \det H$

In our proof-of-concept Tabak–Turner implementation, we do not rely on SGD for training. Instead, we optimize using a Newton-style update based on explicit closed-form expressions for the first- and second-order terms of the log-likelihood. At each step, the update is chosen as

$$\beta^* = \frac{G}{H},$$

where G and H are computed directly from Hessian–vector contractions. Because the update is obtained analytically in this way, we never need to backpropagate through a log-determinant estimator. In our setting, the log-determinant only appears in *monitoring quantities*, such as KL divergence, but it does not drive the optimization itself.

What “Storing Hessian–Vector Products” Means

Stochastic Lanczos Quadrature (SLQ) estimates $\log \det H$ by building a low-dimensional Krylov subspace that approximates the spectrum of the Hessian H . The procedure works as follows:

1. Start with a random probe vector v_1 .
2. Repeatedly apply the Hessian to generate new vectors:

$$v_{k+1} = H v_k.$$

3. Orthogonalize each new vector against the previous ones to form an orthonormal basis $\{q_1, q_2, \dots, q_m\}$ of the Krylov subspace.
4. Project H into this subspace and compute the eigenvalues of the resulting tridiagonal matrix. These eigenvalues are then used in a quadrature formula to approximate $\log \det H$.

Now, consider what happens when we want to *differentiate through SLQ*. Automatic differentiation requires access to every intermediate variable created in the forward pass in order to compute gradients in the backward pass. This means that not only must we compute all Hessian–vector products $\{H v_1, H v_2, \dots, H v_m\}$, but we must also *store them all in memory*. These stored vectors are precisely the Krylov basis vectors.

Why This is Problematic

- **Instability:** Differentiating through the eigendecomposition of T_m is only stable if all eigenvalues are distinct. Coinciding eigenvalues lead to non-unique eigenspaces, producing ill-defined gradients and frequent NaNs.
- **Memory cost:** Storing m basis vectors of dimension n requires $\mathcal{O}(mn)$ memory. For realistic problem sizes, this can reach tens of gigabytes (e.g. > 80 GB for $m = 50$), far larger than the ~ 3 GB required by conjugate-gradient methods.

Why this Causes Memory Blowup

Each vector is n -dimensional, where n is the size of the problem. Storing m such vectors requires $\mathcal{O}(mn)$ memory. For large n and moderate m (e.g. $m = 50$), this becomes extremely costly. As reported in Figure 2 of the paper:

- SLQ with $m = 50$ requires ~ 81 GB,
- SLQ with $m = 20$ requires ~ 32 GB,
- SLQ with $m = 10$ requires ~ 11 GB,
- by contrast, their conjugate-gradient backpropagation method uses only ~ 3 GB.

In short, “storing Hessian–vector products” means storing the entire Krylov basis needed by SLQ. While manageable in the forward pass, this becomes completely impractical once backpropagation forces all of these vectors to be retained in memory.

Forward Pass of SLQ and Memory Usage

The Lanczos procedure underlying SLQ approximates

$$\log \det H$$

for a large symmetric positive-definite matrix H . The algorithm generates an orthonormal Krylov basis

$$Q_m = [q_1, q_2, \dots, q_m], \quad q_i \in \mathbb{R}^n,$$

and constructs the tridiagonal matrix

$$T_m = Q_m^\top H Q_m.$$

Step-By-Step Forward Pass

1. Initialization

Pick a probe vector v_1 and normalize:

$$q_1 = \frac{v_1}{\|v_1\|}.$$

2. Iterative update

For $k = 1, \dots, m$:

$$w_k = H q_k,$$

$$\alpha_k = q_k^\top w_k,$$

$$r_k = w_k - \alpha_k q_k - \beta_{k-1} q_{k-1},$$

$$\beta_k = \|r_k\|,$$

$$q_{k+1} = \frac{r_k}{\beta_k}.$$

The scalars α_k and β_k are stored as entries of the tridiagonal matrix T_m .

Key Observation

Although the Krylov basis Q_m has m vectors in $\mathbb{R}^n$, the recurrence is *three-term*: to produce q_{k+1} , one only needs

$$q_{k-1}, \quad q_k, \quad \text{and } w_k = H q_k.$$

Thus, the forward pass can be implemented while storing only three vectors of length n at any time. The scalars $\{\alpha_k, \beta_k\}$ are recorded to reconstruct T_m , but the full basis $\{q_1, \dots, q_m\}$ never needs to be explicitly kept in memory.

Memory Difference

- **Forward evaluation only.** Requires $\mathcal{O}(n)$ memory: just three vectors in $\mathbb{R}^n$ plus the scalars α_k, β_k . This is why the forward pass of Lanczos is lightweight.
- **Differentiating through SLQ.** Automatic differentiation must reconstruct how each q_k depends on parameters θ . Therefore, it must store *all* intermediate vectors $\{q_1, q_2, \dots, q_m\}$ in memory. This requires $\mathcal{O}(mn)$ memory, which for large n and even moderate m (e.g. $m = 50$) can blow up to tens of gigabytes.

Instead, it is possible to construct an alternative formulation of the gradient as the solution of a convex optimization problem, foregoing the necessity of differentiating through an estimation routine of the log determinant. We adapt the gradient formula from [Chen et al. \(2019a, Appendix C\)](#) to the context of convex potentials. Using Jacobi’s formula* and the adjugate representation of the matrix inverse[†], for any invertible matrix H with parameter θ , we have the following identity:

$$\frac{\partial}{\partial \theta} \log \det H = \frac{1}{\det H} \frac{\partial}{\partial \theta} \det H \stackrel{*}{=} \frac{1}{\det H} \operatorname{tr} \left(\operatorname{adj}(H) \frac{\partial H}{\partial \theta} \right) \stackrel{\dagger}{=} \operatorname{tr} \left(H^{-1} \frac{\partial H}{\partial \theta} \right) = \mathbb{E}_v \left[v^\top H^{-1} \frac{\partial H}{\partial \theta} v \right]. \quad (3)$$

Notably, in the last equality, we used the Hutchinson trace estimator ([Hutchinson, 1989](#)) with a Rademacher random vector v , leading to a $\mathcal{O}(1)$ -memory, unbiased Monte Carlo gradient estimator.

Computing the quantity $v^\top H^{-1}$ in eq. (3) by constructing and inverting the full Hessian requires d calls to an automatic differentiation routine and is too costly for our purposes. However, we can recast this quantity as the solution of a quadratic optimization problem

$$\arg \min_z \left\{ \frac{1}{2} z^\top H z - v^\top z \right\} \quad (4)$$

which has the unique minimizer $z^* = H^{-1}v$ since H is symmetric positive definite.

Step-by-Step Derivation: Gradient of $\log \det H(\theta)$ and the Quadratic Reformulation

Setup

Let $H(\theta) \in \mathbb{R}^{d \times d}$ be differentiable in θ and symmetric positive definite (SPD) for all θ considered. We want an efficient, low-memory way to compute

$$\nabla_\theta \log \det H(\theta).$$

From log det to a trace (show every step)

Start with Jacobi’s formula for the determinant:

$$\frac{\partial}{\partial \theta} \det H = \operatorname{tr} \left(\operatorname{adj}(H) \frac{\partial H}{\partial \theta} \right).$$

Use the identity $\operatorname{adj}(H) = (\det H) H^{-1}$ (valid when H is invertible):

$$\frac{1}{\det H} \frac{\partial}{\partial \theta} \det H = \operatorname{tr} \left(H^{-1} \frac{\partial H}{\partial \theta} \right).$$

Recognize the left side as $\frac{\partial}{\partial \theta} \log \det H$:

$$\frac{\partial}{\partial \theta} \log \det H = \operatorname{tr} \left(H^{-1} \frac{\partial H}{\partial \theta} \right) \quad (A)$$

Hutchinson’s Trace Estimator (Unbiasedness Shown Explicitly)

Let $A \in \mathbb{R}^{d \times d}$ be arbitrary and let $v \in \{\pm 1\}^d$ have i.i.d. Rademacher entries with $\mathbb{E}[v] = 0$ and $\mathbb{E}[vv^\top] = I$.

$$\mathbb{E}_v [v^\top A v] = \mathbb{E}_v [\operatorname{tr}(v^\top A v)] = \mathbb{E}_v [\operatorname{tr}(A v v^\top)] = \operatorname{tr}(A \mathbb{E}_v [v v^\top]) = \operatorname{tr}(A).$$

Derivation of Hutchinson's Identity

We want to show that

$$\mathbb{E}_v[v^\top Av] = \text{tr}(A), \quad (\text{B})$$

where $A \in \mathbb{R}^{d \times d}$ and $v \in \{\pm 1\}^d$ is a random vector with i.i.d. Rademacher entries.

Express the Scalar as a Trace

A scalar can be written as the trace of itself:

$$v^\top Av = \text{tr}(v^\top Av).$$

This is valid since $v^\top Av \in \mathbb{R}$ is 1×1 , and the trace of a 1×1 matrix is just that scalar.

Rearrangement using the cyclic property of trace

Recall the *cyclic property of the trace*: for any conformable matrices X, Y ,

$$\text{tr}(XY) = \text{tr}(YX).$$

More generally, for three matrices X, Y, Z of compatible dimensions,

$$\text{tr}(XYZ) = \text{tr}(ZXY) = \text{tr}(YZX).$$

Note: conformable matrices and compatible dimensions simply means the matrices are of the right sizes so that multiplication makes sense. Now apply this to $X = v^\top$, $Y = A$, and $Z = v$:

$$\text{tr}(v^\top Av) = \text{tr}(Avv^\top).$$

Explanation: The quantity $v^\top Av$ is a 1×1 scalar, which we may write as $(v^\top A)v$. Using the cyclic property of the trace, we can rotate the factors inside without changing the value, which allows us to move the rightmost v to the front. This produces the expression $Avv^\top$ inside the trace, thereby converting the scalar quadratic form into a trace involving the rank-one matrix $vv^\top$:

$$v^\top Av = \text{tr}(Avv^\top).$$

Apply expectation

Taking expectation and using linearity of both trace and expectation:

$$\mathbb{E}_v[v^\top Av] = \mathbb{E}_v[\text{tr}(Avv^\top)] = \text{tr}(A \mathbb{E}_v[vv^\top]).$$

Note: A is constant with respect to the expectation

Compute $\mathbb{E}[vv^\top]$

For a Rademacher vector v : - $\mathbb{E}[v_i] = 0$, - $\mathbb{E}[v_i^2] = 1$, - $\mathbb{E}[v_i v_j] = 0$ for $i \neq j$. Thus,

$$\mathbb{E}[vv^\top] = I.$$

Final simplification

Substitute back:

$$\text{tr}(A \mathbb{E}[vv^\top]) = \text{tr}(AI) = \text{tr}(A).$$

Final Result

$$\mathbb{E}_v[v^\top A v] = \text{tr}(A).$$

Apply (B) to $A = H^{-1} \frac{\partial H}{\partial \theta}$ and combine with (A):

$$\frac{\partial}{\partial \theta} \log \det H = \mathbb{E}_v \left[v^\top H^{-1} \frac{\partial H}{\partial \theta} v \right] \quad (\text{B})$$

Thus a Monte Carlo estimator with S samples $\{v^{(s)}\}_{s=1}^S$ is

$$\frac{\partial}{\partial \theta} \widehat{\log \det H} = \frac{1}{S} \sum_{s=1}^S (v^{(s)})^\top H^{-1} \frac{\partial H}{\partial \theta} v^{(s)}.$$

Eliminate Explicit Inverses by Solving Linear Systems

For each sample v , define $z := H^{-1}v$. Equivalently,

$$Hz = v.$$

Then the inside of (C) becomes

$$v^\top H^{-1} \frac{\partial H}{\partial \theta} v = \underbrace{(H^{-1}v)^\top}_z \frac{\partial H}{\partial \theta} v = z^\top \frac{\partial H}{\partial \theta} v.$$

Quadratic Optimization Reformulation (Equivalence)

Define the quadratic function

$$\phi(z) = \frac{1}{2} z^\top H z - v^\top z,$$

which is strictly convex since $H \succ 0$. Its gradient and Hessian are

$$\nabla_z \phi(z) = H z - v, \quad \nabla_z^2 \phi(z) = H \succ 0.$$

The stationary point satisfies $H z - v = 0$, i.e. $H z = v$. Thus the optimizer is

$$z^* = \arg \min_z \left\{ \frac{1}{2} z^\top H z - v^\top z \right\} = H^{-1}v.$$

Note: Introducing the substitution $z := H^{-1}v$ is a computational trick. Instead of forming H^{-1} explicitly (which is expensive and numerically unstable for large matrices), we solve the linear system $H z = v$ to obtain z . This avoids computing a full inverse and reduces the problem to (i) solving linear systems and (ii) performing matrix-vector products with $\frac{\partial H}{\partial \theta}$.

Componentwise Gradient for Vector Parameters

If $\theta = (\theta_1, \dots, \theta_p)$ and $H = H(\theta)$, then for each k :

$$\frac{\partial}{\partial \theta_k} \log \det H = \mathbb{E}_v \left[v^\top H^{-1} \left(\frac{\partial H}{\partial \theta_k} \right) v \right] = \mathbb{E}_v \left[z^\top \left(\frac{\partial H}{\partial \theta_k} \right) v \right], \quad \text{with } z = H^{-1}v.$$

Practical Estimator (Explicit Steps)

Given S probe vectors:

1. For $s = 1, \dots, S$, sample $v^{(s)} \in \{\pm 1\}^d$ i.i.d.

2. For each s , compute $z^{(s)}$ by solving the SPD system

$$Hz^{(s)} = v^{(s)} \quad (\text{or, equivalently, minimize } \frac{1}{2}(z^{(s)})^\top Hz^{(s)} - (v^{(s)})^\top z^{(s)}).$$

3. For each parameter component θ_k , form the scalar

$$g_k^{(s)} = (z^{(s)})^\top \left(\frac{\partial H}{\partial \theta_k} \right) v^{(s)}.$$

4. Output the Monte Carlo estimate

$$\frac{\partial}{\partial \theta_k} \widehat{\log \det H} = \frac{1}{S} \sum_{s=1}^S g_k^{(s)}.$$

Computational Notes (Why this is Memory-Efficient)

- No explicit inverse is formed. Each $z^{(s)}$ is obtained by an iterative solver using *matrix-vector products* with H .
- The Hutchinson estimator needs only the current $v^{(s)}$, $z^{(s)}$, and the product $\left(\frac{\partial H}{\partial \theta_k} \right) v^{(s)}$, yielding $\mathcal{O}(1)$ additional memory beyond vectors.
- SPD of H guarantees the unique minimizer in (F) and enables fast convergence of Conjugate Gradient.

Final Identity (Concise)

$$\frac{\partial}{\partial \theta_k} \log \det H(\theta) = \mathbb{E}_v \left[(H^{-1}v)^\top \left(\frac{\partial H}{\partial \theta_k} \right) v \right], \quad \text{with } H^{-1}v \text{ obtained via } \arg \min_z \{ \frac{1}{2} z^\top H z - v^\top z \}.$$

We use the *conjugate gradient* (CG) method, which is specifically designed for solving the unconstrained optimization problems in eq. (4) with symmetric positive definite H . It uses only Hessian-vector products and is straightforward to parallelize. Conjugate gradient is guaranteed to return the exact solution z^* within d iterations, and the error of the approximation is known to converge exponentially fast $\|z^m - z^*\|_H \leq 2\gamma^m \|z^0 - z^*\|_H$, where z^m is the estimate after m iterations. The rate of convergence $\gamma < 1$ relates to the condition number of H . For more details, see Nocedal & Wright (2006, Ch. 5). In practice, we terminate CG when $\|Hz^m - v\|_\infty < \tau$ is satisfied for some user-controlled tolerance. Empirically, we find that stringent tolerance values are unnecessary for stochastic optimization (see appendix F).

Estimating the full quantity in eq. (3) is then simply a matter of computing and differentiating a scalar quantity (a surrogate objective) involving another Hessian-vector product: $\frac{d}{d\theta} ((z^m)^\top H v)$, where only H is differentiated through (since z^m is only used to approximate $v^\top H^{-1}$ as a modifier of the gradient). We summarize this procedure in Algorithm 2. Similar to inversion, the `hvp` can also be computed in batch by summing over the data index, since all entries are independent.

Algorithm 2 Surrogate training objective.

```

1: procedure SURROGATEOBJ( $F, x, \text{CG}$ )
2:   Obtain the gradient  $f(x) \triangleq \nabla_x F(x)$ 
3:   Sample Rademacher random vector  $r$ 
4:   def hvp( $v$ ):
5:     return  $v^\top \frac{\partial}{\partial x} f(x)$ 
6:    $z \leftarrow \text{stop\_gradient}(\text{CG}(\text{hvp}, r))$ 
7:   return  $\text{hvp}(z)^\top r$ 

```

ICNN Architecture Despite the universal property, having a poor parameterization can lead to difficulties in optimization and limit the effective expressivity of the model. We propose an architectural enhancement of ICNN, defined as follows (note the change in notation: instead of writing the pre-activations z , we use h to denote the activated units):

$$F^{aug}(x) := L_{K+1}^+(h_K) + L_{K+1}(x)$$

$$h_k := \text{concat}([\tilde{h}_k, h_k^{aug}]) \quad \tilde{h}_k := s(L_k^+(h_{k-1}) + L_k(x)) \quad h_k^{aug} = s(L_k^{aug}(x)) \quad (7)$$

where half of the hidden units are directly connected to the input, so the gradient would have some form of skip connection. We call this the input-augmented ICNN. Unless otherwise stated, we use the input-augmented ICNN as the default architecture.